

Белорусский государственный университет
Факультет прикладной математики и информатики

Кафедра компьютерных
технологий и систем

Ю.В. Буяльская, В.В. Казаченок

**ВВЕДЕНИЕ В КОМПЬЮТЕРНЫЙ И
ИНТЕЛЛЕКТУАЛЬНЫЙ АНАЛИЗ ДАННЫХ**

Методические указания
для студентов факультета прикладной математики и
информатики

МИНСК

Р е ц е н з е н т

доцент кафедры математического моделирования и анализа данных, кандидат физико-математических наук *В. И. Малюгин*

Рассматриваются теоретические основы интеллектуального анализа данных, включающие характеристику основных этапов анализа и начальные сведения об используемом математическом аппарате. Подробно описываются возможности современной программной среды R анализа данных.

Предназначено для студентов факультета прикладной математики и информатики, обучающихся по специальности «Информатика».

ОГЛАВЛЕНИЕ

1	Теоретические основы интеллектуального анализа данных.....	4
1.1	Что такое «Интеллектуальный анализ данных».....	4
1.2	Типы статистических данных.....	7
1.3	Числовые характеристики вариационных рядов.....	10
1.4	Корреляционный анализ	14
1.5	Регрессионный анализ	17
1.6	Ряды динамики	18
1.7	Кластерный анализ.....	20
1.8	Дискриминантный анализ.....	23
2	Основы компьютерного анализа данных	26
2.1	Введение в R.....	26
2.2	Форматы данных	27
2.3	Управляющие конструкции и функции	34
2.4	Статистические функции	35
2.5	Графики	36
2.6	Кластерный анализ.....	39
2.7	Дискриминантный анализ.....	41
2.8	Лабораторные работы	43

1 Теоретические основы интеллектуального анализа данных

1.1 Что такое «Интеллектуальный анализ данных»

Интеллектуальный анализ данных (ИАД или data mining) – это совокупность математических моделей, численных методов, программных средств и информационных технологий, обеспечивающих обнаружение в эмпирических данных доступной для интерпретации информации и синтез на основе этой информации ранее неизвестных, нетривиальных и практически полезных для достижения определенных целей знаний.

ИАД – это мультидисциплинарная область, схематическое изображение которой приведено на рис. 1.

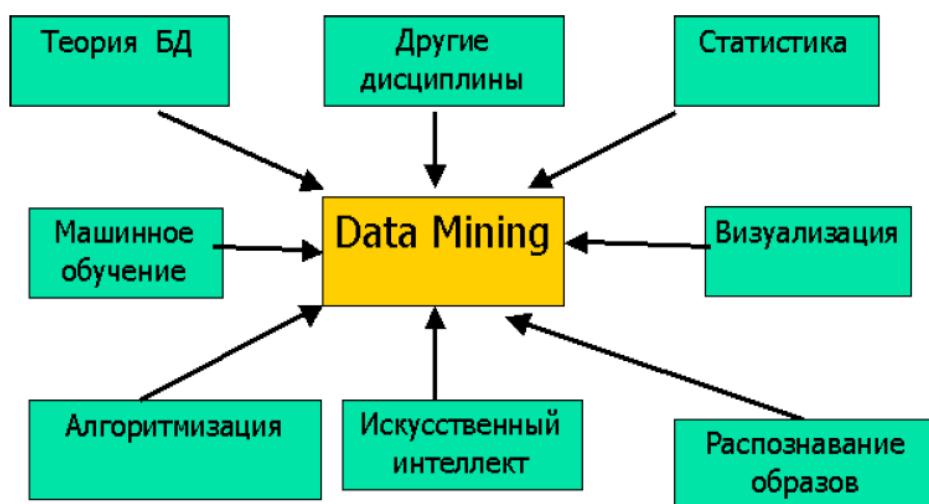


Рис. 1 – Схематическое изображение data mining

Здесь под другими дисциплинами понимаются, в первую очередь, дисциплины, связанные с предметной областью исследований.

Системы интеллектуального анализа данных – класс программных систем поддержки принятия решений в больших объемах разнородных, сложно структурированных данных.

Процесс ИАД состоит из следующих этапов:

➤ Анализ предметной области:

- ✓ выявление и формулировка необходимых априорных знаний о предметной области, целей анализа, задач приложения, сценариев использования.

➤ Формирование и подготовка данных для анализа:

- ✓ поиск (или выбор) «сырых» данных, возможно реализация подсистемы сбора (консолидации);
- ✓ предобработка данных (нормализация, дискретизация, обработка пропущенных значений, удаление артефактов, проверка консистентности);
- ✓ уменьшение размерности, выбор значимых характеристик, расчет интегральных показателей и инвариантов.
- Определение типа решаемой задачи анализа:
 - ✓ классификация, прогнозирование, кластеризация, поиск исключений, ассоциативный анализ и т.д.
- Выбор (или разработка) алгоритма анализа:
 - ✓ определение ограничений и требований к алгоритму по точности, размеру, интерпретируемости, скорости построения и применения получаемых моделей, по типу исходных данных.
- Собственно «Data mining»:
 - ✓ применение выбранного алгоритма анализа для поиска закономерностей выбранного типа и построение моделей.
- Проверка моделей и представление результатов анализа:
 - ✓ визуализация, преобразование, удаление избыточности, оценка точности, достоверности моделей и т.д.
- Применение построенных моделей:
 - ✓ Descriptive data mining – информирование аналитика, «описательные» модели; основная цель – визуализация;
 - ✓ Predictive data mining – прогнозирование неизвестных значений или характеристик в «новых» данных с помощью построенных моделей; основная цель – прогноз.

Приведенные этапы можно изобразить графически в виде схемы, приведенной на рис. 2.

Отличия ИАД систем от других систем обработки и анализа данных заключаются в следующем:

- Наличие «обучения»:
 - ✓ база знаний формируются на основе анализируемых данных, а не экспертных знаний (в отличие от традиционных экспертных систем и систем информационного поиска);
 - ✓ структура модели и искомые зависимости заранее не известны (в отличие от статистических пакетов, ориентированных на расчет статистик, проверку гипотез и оценку параметров распределений).
- Наличие большого объема данных сложной структуры:

- ✓ зачастую скорость работы алгоритмов в ИАД важнее отклонений по точности (“quick and dirty solution”);
- ✓ большинство алгоритмов работают с исходными данными в виде числовой матрицы признаков: сложная структура реальных объектов в ИАД приводит к необходимости решать задачу построения пространства характеристик и отображения в него свойств исходных объектов;
- ✓ перечисленные выше особенности отличают ИАД системы от традиционных систем машинного обучения, в которых, как правило, решается обратная задача – построение достоверной модели в условиях малой обучающей выборки.

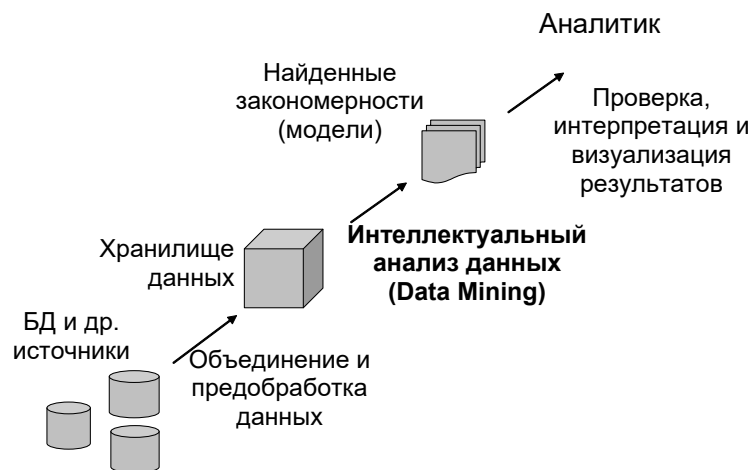


Рис. 2 – Процесс ИАД

➤ Наличие человека - аналитика как окончательного потребителя результатов работы ИАД системы:

- ✓ в сценарии работы любой системы ИАД всегда присутствует аналитик, даже если полученная в результате модель далее используется для автоматической классификации;
- ✓ аналитик формирует тренировочные наборы, производит настройку алгоритмов, обучение и дообучение, анализирует полученные модели и принимает решения об их дальнейшем использовании;
- ✓ таким образом, системы автоматической классификации, кластеризации и распознавания образов, даже использующие возможность дообучения, не являются системами ИАД.

Математический аппарат

При выявлении различных закономерностей в системах *data mining* применяются следующие методы: деревья решений; алгоритмы кластеризации; регрессионный анализ; нейронные сети; временные ряды и др.

Типы выявляемых закономерностей

Системы *data mining* обеспечивают выявление скрытых закономерностей различных видов. Перечислим некоторые из них:

✓ *Ассоциация (идентификация)*. Если некоторый факт-1 является частью определенного события, то с расчетной вероятностью и другой факт-2, связанный с первым, будет частью того же события.

✓ *Последовательность (прогнозирование)*. Если свершилось некоторое событие-1, то с расчетной вероятностью через определенный период времени свершится другое событие-2, связанное с первым.

✓ *Классификация*. На основании информации о свойствах объекта ему присваивается определенное дискретное значение показателя, по которому проводится классификация (идентификатор).

✓ *Кластеризация*. Наиболее сходные по своим признакам объекты объединяются в группы (кластеры) таким образом, что в разных кластерах оказываются наиболее сильно отличающиеся друг от друга объекты. Кластеризация аналогична классификации, но в отличие от последней классы – кластеры объектов заранее не известны, а формируются в процессе кластеризации.

В целом, прошлые фактические значения величин используются для прогнозирования будущих значений тех же или других величин на основании знания зависимостей между ними, трендов и статистики.

1.2 Типы статистических данных

Категории статистики – статистическая совокупность, единица совокупности, признак, статистический показатель.

Статистическая совокупность – совокупность изучаемых объектов или явлений, имеющих общую качественную основу, но отличающихся друг от друга отдельными признаками.

Единица совокупности – первичный элемент статистической совокупности, являющийся носителем признаков, подлежащих регистрации. Единица совокупности рассматривается как неделимый элемент.

Признак – показатель, характеризующий индивидуальную особенность единицы совокупности, рассматриваемый как случайная величина. Значение признака – измеренный индивидуальный показатель.

Классификация признаков приведена на рис. 3.



Рис. 3 – Классификация признаков статистической совокупности

Примеры типовых измерительных шкал:

➤ Атрибутивные (качественные)

- ✓ Шкала наименований (напр.: имя, пол, семейство, класс, номер игрока ...);
- ✓ Порядковая (ранговая) шкала (напр.: ранг служащего, балльные шкалы (сила ветра, оценка на экзамене, магнитуда землетрясения, твердость минерала) ...).

➤ Количественные

- ✓ Интервальная шкала (напр.: температура °С, °F, летоисчисление, высота над уровнем моря ...);
- ✓ Шкала отношений (напр.: длина, высота, вес, скорость, светимость ...).

Статистический показатель – количественно-качественная обобщающая характеристика какого-либо свойства группы (части) единиц совокупности или совокупности в целом.

Генеральная совокупность – исследуемая статистическая совокупность.

Выборочная совокупность – отобранные единицы генеральной совокупности, «выборка». *Представительность выборки*

(репрезентативность) – близость свойств генеральной и выборочной совокупностей.

Статистическое наблюдение – планомерный, научно организованный сбор информации о массовых явлениях путем регистрации заранее намеченных признаков с целью получения обобщающих характеристик.

Виды статистических наблюдений по охвату единиц совокупности:

- Сплошное: все единицы;
- Несплошное: часть единиц.

Если ряд распределения построен по количественному признаку, то такой ряд называют **вариационным**.

Построить вариационный ряд – значит *упорядочить* количественное распределение единиц совокупности по значениям признака, а затем подсчитать частоту единиц совокупности с этими значениями (построить групповую таблицу).

Пример дискретного ряда и соответствующих ему вариационных рядов приведен на рис. 4.

В магазине продана мужская обувь следующих размеров: 38, 41, 41, 38, 43, 39, 39, 42, 42, 39, 42, 39, 40, 40, 40, 39, 39.						
Дискретный вариационный ряд:						
Размер обуви	38	39	40	41	42	43
Кол-во пар	2	6	3	2	3	1
Интервальный вариационный ряд:						
Размеры обуви	38-39		40-41		42-43	
Кол-во пар	8		5		4	

Рис. 4 – Примеры построения вариационных рядов

Если за основу группировки взят *качественный* признак, то такой ряд распределения называют **атрибутивным** (например, распределение по видам труда, по полу, по профессии, по религиозному признаку, национальной принадлежности и т.д.).

Статистическая группировка формально-математическим способом предполагает использование формулы Стерджесса (1):

$$k = 1 + [\log_2 n], \quad (1)$$

где n – число единиц совокупности; k – число групп.

1.3 Числовые характеристики вариационных рядов

Основными характеристиками «середины» распределения являются: среднее значение (mean), медиана (median), мода (mode).

Медиана – значение, которое делит дискретный ряд пополам: половина значений (вариант) больше медианы, половина – меньше.

Если дискретный ряд содержит *нечетное* количество вариантов, то медианой является значение той единственной варианты, справа и слева от которой находится одинаковое число вариантов.

Если дискретный ряд содержит *четное* количество вариантов, то находятся две варианты, справа и слева от которых располагается одинаковое количество вариантов. И медиана равна среднему арифметическому этих двух значений.

Дискретный ряд можно поделить не только на две равные части, но и на четыре (значения, стоящие на границах – квантили):

Квантили (quartiles) делят дискретный ряд на четыре части так, что в каждой из них оказывается поровну значений (2-я квантиль – медиана).

Интерквартильный размах – разность между третьей и первой квантилями.

Размах вариации – разность между наибольшим и наименьшим значениями дискретного ряда.

Мода — это варианта, которая имеет наибольшую частоту.

Соглашения о существовании моды:

Если все варианты наблюдаются с одинаковой частотой, то говорят, что вариационный ряд не имеет моды.

Если две или более соседние варианты имеют наибольшие частоты, равные между собой, то мода равна средней арифметической этих вариантов.

Если равные варианты, имеющие наибольшие частоты, расположены не по соседству, то принято говорить, что признак имеет две и более моды (бимодальный, полимодальный признаки и т.д.).

В качестве **среднего значения** чаще всего выступает среднее арифметическое.

Под *средним арифметическим* понимается такое значение признака, которое бы имела каждая единица совокупности, если бы общий итог всех значений признака $x_i, i = 1, 2, \dots, n$, был распределён равномерно между всеми единицами совокупности.

Простое среднее арифметическое вычисляется по следующей формуле (2):

$$\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i . \quad (2)$$

Для вариационного ряда взвешенное среднее арифметическое вычисляется по формуле (3):

$$\bar{x} = \frac{1}{n} \sum_{i=1}^k m_i x_i , \quad (3)$$

где $n = \sum_{i=1}^k m_i$, m_i – число повторений признака x_i , k – количество различных значений признака.

Мода, медиана и среднее совпадают для симметричного унимодального распределения.

При сборе данных часто встречаются наблюдения, резко отличающиеся по величине от большинства вариантов, которые называются *аномальными вариантами*.

Средние величины, которые сохраняют точность представления выборки при наличии аномальных наблюдений, называются *робастными*.

Например, *средняя (усеченная порядка α)* для упорядоченных значений признака x_i , $i = 1, 2, \dots, n$, вычисляется по формуле (4):

$$\bar{x}_\alpha = \frac{1}{n-2m} (x_{(m+1)} + x_{(m+2)} + \dots + x_{(n-m)}) , \quad (4)$$

где $\alpha = \frac{m}{n}$ – доля «ненадежных» вариантов.

«Ширину» распределения характеризует дисперсия.

Дисперсия – это среднее арифметическое квадратов отклонений каждого значения признака от общего среднего.

Как и среднее арифметическое, дисперсия может быть вычислена, в зависимости от способа представления исходных данных, по следующим формулам (5)-(6):

$$\sigma^2 = \frac{1}{n} \sum_{i=1}^n (x_i - \bar{x})^2 , \quad (5)$$

или

$$\sigma^2 = \frac{1}{n} \sum_{i=1}^k m_i (x_i - \bar{x})^2 , \quad n = \sum_{i=1}^k m_i . \quad (6)$$

В практических расчетах дисперсии часто используется следующая формула (7):

$$\sigma^2 = \frac{1}{n} \sum_{i=1}^n x_i^2 - (\bar{x})^2 = \overline{x^2} - (\bar{x})^2 . \quad (7)$$

Среднее квадратическое отклонение представляет собой корень квадратный из дисперсии (8):

$$\sigma = \sqrt{\sigma^2} . \quad (8)$$

Коэффициент вариации, характеризующий степень однородности выборки, вычисляется по формуле (9):

$$V = \frac{\sigma}{x} \cdot 100\% . \quad (9)$$

При этом, если $V < 33\%$, то выборка считается однородной; если $33\% < V < 100\%$, то выборка имеет допустимую степень концентрации. При $V > 100\%$ выборка считается неоднородной, и нет смысла вычислять многие из вышеприведенных числовых характеристик.

Для графического изображения дискретных вариационных рядов используют *гистограммы, полигоны частот и кумуляты*. Их примеры приведены на рис. 5 – 8.

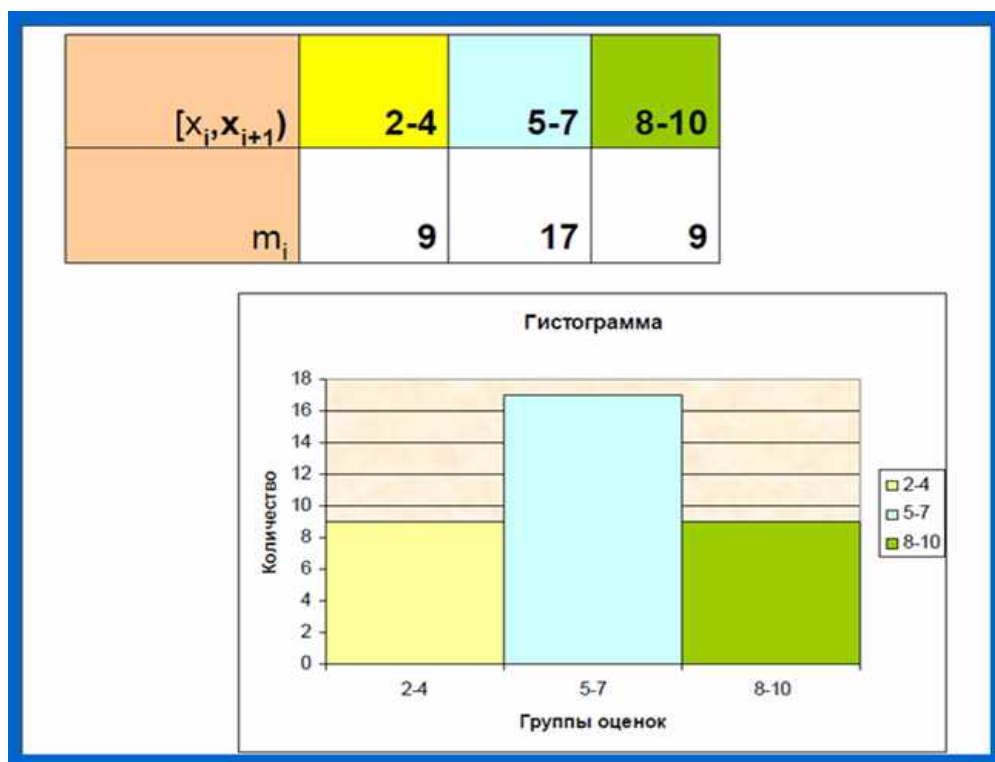


Рис. 5 – Пример гистограммы



Рис. 6 – Пример полигона частот

Оценки	Количество	Накопл. абс. частоты	Накопл. отн. частоты	в %
3	5	5	0,13	13%
4	6	11	0,28	28%
5	7	18	0,46	46%
6	9	27	0,69	69%
7	5	32	0,82	82%
8	4	36	0,92	92%
9	2	38	0,97	97%
10	1	39	1,00	100%

Рис. 7 – Пример данных для кумуляты

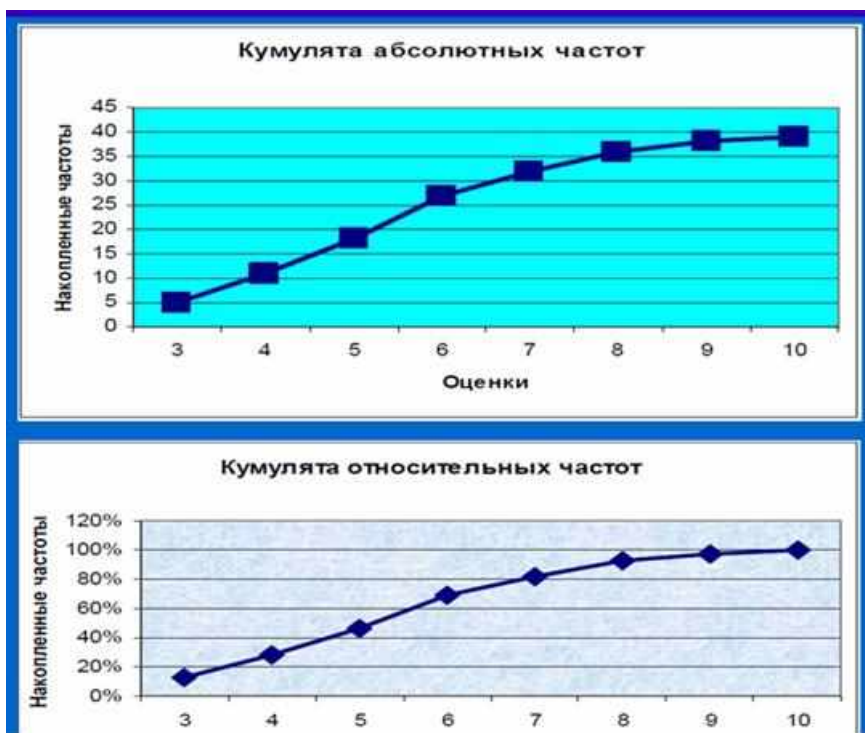


Рис. 8 – Пример кумуляты

1.4 Корреляционный анализ

Для измеренных значений пар признаков $(x_i; y_i)$, $i = 1, 2, \dots, n$, силу связи между x_i и y_i можно определить с использованием *линейного (парного) коэффициента корреляции*.

1). Количественная шкала. Если оба признака измерены в количественных шкалах, то вычисление коэффициента корреляции проводится по формуле (10):

$$r_{X,Y} = \frac{1}{n} \sum_{i=1}^n \left(\frac{x_i - \bar{x}}{\sigma_X} \right) \left(\frac{y_i - \bar{y}}{\sigma_Y} \right) = \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_{i=1}^n (x_i - \bar{x})^2 \sum_{i=1}^n (y_i - \bar{y})^2}}, \quad (10)$$

$$\text{где } \sigma_X = \sqrt{\frac{1}{n} \sum_{i=1}^n (x_i - \bar{x})^2}, \quad \sigma_Y = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \bar{y})^2}.$$

Если среди всех возможных значений признаков x_i и y_i ($i = 1, 2, \dots, n$) выделить только различные значения (k и l значений соответственно), то двумерное распределение частот можно представить в виде таблицы:

X \ Y	y_1	y_2	...	y_l	m_{i*}
x_1	m_{11}	m_{12}	...	m_{1l}	m_{1*}

x_2	m_{21}	m_{22}	...	m_{2l}	m_{2*}
...	...	...	...	...	...
x_k	m_{k1}	m_{k2}	...	m_{kl}	m_{k*}
m_{*j}	m_{*1}	m_{*2}	...	m_{*l}	n

Здесь m_{ij} – частота наблюдаемой пары (x_i, y_j) , $n = \sum_{i=1}^k \sum_{j=1}^l m_{ij}$, $m_{i*} = \sum_{j=1}^l m_{ij}$,

$$m_{*j} = \sum_{i=1}^k m_{ij}.$$

В этом случае линейный коэффициент корреляции вычисляется по формуле (11):

$$r_{X,Y} = \frac{s_{xy}}{\sigma_x \sigma_y}, \quad (11)$$

где σ_x , σ_y – среднее квадратическое отклонение признаков x и y соответственно, а s_{xy} – ковариация признаков x и y , определяемая формулой

$$s_{xy} = \frac{1}{n} \sum_{i=1}^k \sum_{j=1}^l m_{ij} (x_i - \bar{x}) (y_j - \bar{y}).$$

Коэффициент корреляции принимает значения от -1 до $+1$. При этом положительные значения коэффициента указывают на наличие прямо пропорциональной зависимости между признаками x и y , а отрицательные значения – обратно пропорциональной зависимости.

Приблизительно силу связи можно определить с помощью приводимой на рис. 9 шкалы Чеддока.

Для проверки значимости коэффициента корреляции можно вычислить (12):

$$t_{расч} = |r_{xy}| \sqrt{\frac{n-2}{1-r_{xy}^2}}, \quad (12)$$

которое сравнивается с табличным значением $t_{табл}$ t -распределения Стьюдента с числом степеней свободы $n-2$.

Если $t_{расч} > t_{табл}$, то существует зависимость между признаками x и y , $t_{расч} \leq t_{табл}$ – признаки x и y независимы.

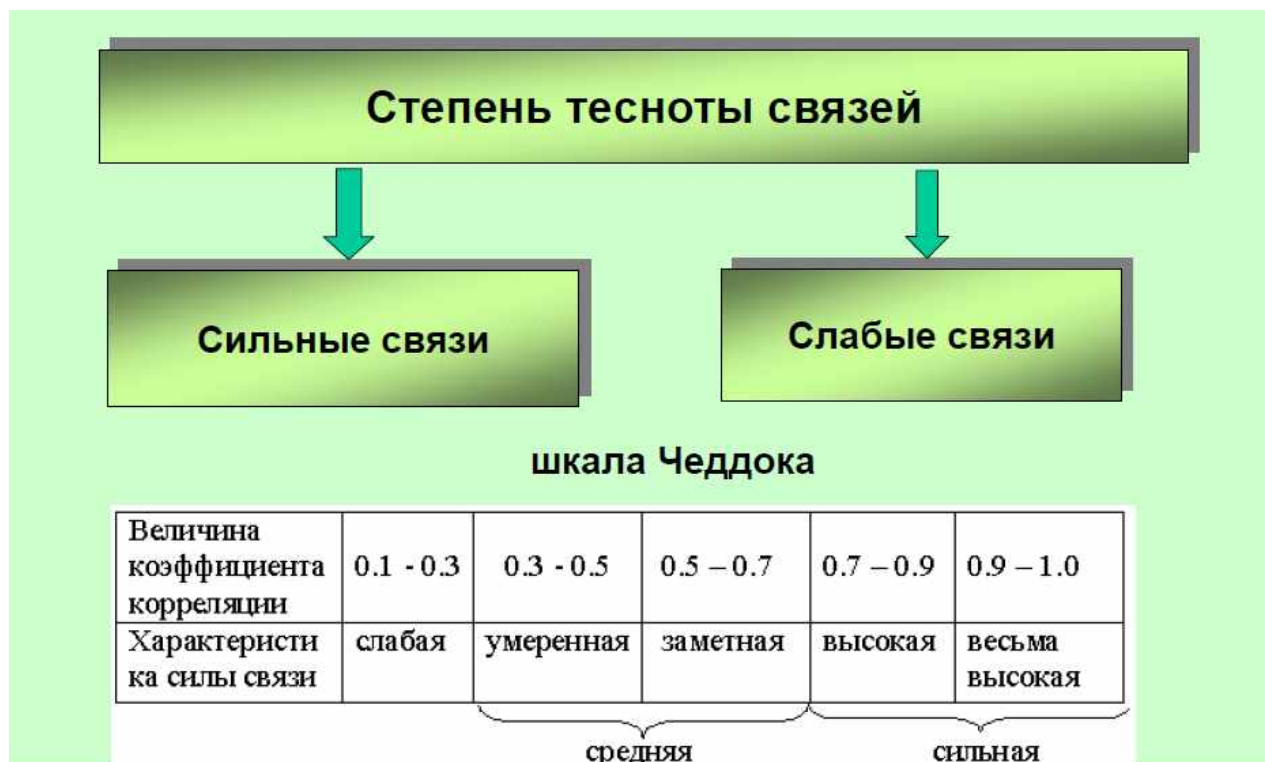


Рис. 9 – Шкала силы связи

2). Ранговая (порядковая) шкала. Пусть признаки измерены в ранговой шкале $P = \{p_i, i = \overline{1, n}\}$ и $Q = \{q_i, i = \overline{1, n}\}$

Тогда коэффициент корреляции можно вычислить по формуле Спирмена (13):

$$\rho = 1 - 6 \frac{\sum_{i=1}^n (p_i - q_i)^2}{n(n^2 - 1)}, \quad (13)$$

3). Номинальная (качественная) шкала. Если оба признака измерены в дихотомической шкале (т.е. принимают значения 0 или 1), то коэффициент корреляции можно вычислить по формуле Пирсона (14):

$$\varphi = \frac{p(x, y) - p(x)p(y)}{\sqrt{p(x)(1 - p(x))p(y)(1 - p(y))}}, \quad (14)$$

где $p(x)$ – доля выборочных значений признака x , равных 1, $p(y)$ – доля выборочных значений признака y , равных 1, $p(x, y)$ – доля вариантов $(x_i; y_i)$ с единичными значениями у обоих признаков.

1.5 Регрессионный анализ

Пусть задана выборка $(x_i; y_i)$, $i = 1, 2, \dots, n$. Рассмотрим способ определения оценок a и b эмпирического уравнения регрессии

$$\hat{y}_i = a + bx_i, i = 1, 2, \dots, n,$$

где $\hat{y}_i$ – оценка условного математического ожидания $M(Y | X = x_i)$; a и b – оценки неизвестных параметров регрессии.

На рис. 10 выборка $(x_i; y_i)$, $i = 1, 2, \dots, n$, изображена точками в системе координат XOY ; $Y = a + bX$ – уравнение изображенной прямой.

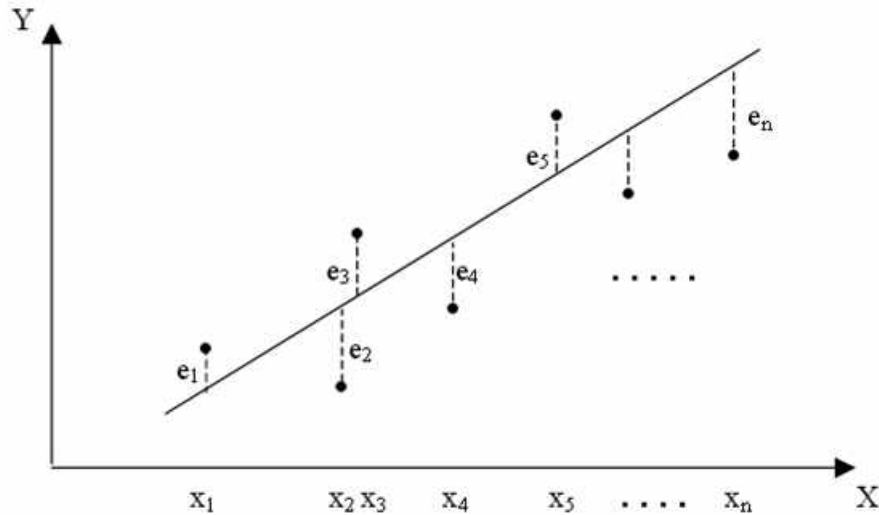


Рис. 10 – Графическая интерпретация регрессионной модели

Следовательно, в конкретном случае, $y_i = a + bx_i + e_i$, $i = 1, 2, \dots, n$, где отклонение e_i – оценка случайного отклонения.

Метод наименьших квадратов (МНК) является самым распространенным и теоретически обоснованным методом нахождения неизвестных параметров (коэффициентов) a и b , сущность которого заключается в минимизации суммы квадратов отклонений (15):

$$S = \sum_{i=1}^n e_i^2 = \sum_{i=1}^n (y_i - \hat{y}_i)^2 = \sum_{i=1}^n (y_i - a - bx_i)^2. \quad (15)$$

Оценки a и b , доставляющие минимум S , могут быть вычислены по формулам:

$$a = \frac{\sum_{i=1}^n y_i - b \sum_{i=1}^n x_i}{n} = \bar{y} - b\bar{x}, \quad b = \frac{\frac{\sum_{i=1}^n x_i \sum_{i=1}^n y_i}{n} - \sum_{i=1}^n x_i y_i}{\left(\frac{\sum_{i=1}^n x_i}{n} \right)^2 - \sum_{i=1}^n x_i^2} \quad \text{или} \quad b = r_{xy} \frac{\sigma_y}{\sigma_x}$$

Рассматриваемая регрессионная модель $y_i = a + bx_i + e_i$, $i = 1, 2, \dots, n$, может быть представлена в матричном виде (16):

$$Y = X\beta + E, \quad (16)$$

где

$$Y = \begin{pmatrix} y_1 \\ \dots \\ y_n \end{pmatrix}, \quad E = \begin{pmatrix} e_1 \\ \dots \\ e_n \end{pmatrix}, \quad \beta = \begin{pmatrix} a \\ b \end{pmatrix}, \quad X = \begin{pmatrix} 1 & x_1 \\ \dots & \dots \\ 1 & x_n \end{pmatrix}.$$

Тогда оценки неизвестных параметров a и b методом наименьших квадратов вычисляются по формуле (17):

$$\hat{\beta} = (X^T X)^{-1} X^T Y. \quad (17)$$

Приведенная матричная модель может быть легко обобщена на многомерный случай, если $x_1, x_2, \dots, x_n$ рассматривать в матрице X как векторы-строки.

1.6 Ряды динамики

Ряды динамики – статистические данные, отображающие развитие во времени изучаемого явления. Их также называют *динамическими рядами*, *временными рядами*.

В каждом ряду динамики имеется два основных элемента:

- 1) показатель времени – это моменты или периоды времени, к которым относятся числовые значения показателей;
- 2) соответствующие моментам времени уровни развития изучаемого явления.

Всякий ряд динамики может быть представлен в виде составляющих:

- * тренд – основная тенденция развития динамического ряда (к увеличению или снижению его уровней);
- * циклические (периодические) колебания, в том числе сезонные;
- * случайные колебания.

Рассмотрим основные методы выделения тренда.

1. Метод укрупнения интервалов.

Ряд динамики разделяют на некоторое достаточно большое число равных интервалов. Средние уровни, рассчитанные по укрупненным интервалам, позволяют увидеть тенденцию развития явления.

2. Метод скользящей средней.

В этом методе исходные уровни ряда заменяются средними величинами, которые получают из данного уровня и нескольких симметрично его окружающих.

Формулы расчета по скользящей средней выглядят, в частности при сглаживании по 3 уровням, следующим образом (18):

$$\bar{Y}_i = \frac{Y_{i-1} + Y_i + Y_{i+1}}{3} . \quad (18)$$

При сглаживании по трем точкам выровненное значение в начале ряда рассчитывается по формуле (19):

$$\bar{Y}_1 = \frac{5Y_1 + 2Y_2 - Y_3}{6} . \quad (19)$$

Для последней точки расчет симметричен (20):

$$\bar{Y}_n = \frac{5Y_n + 2Y_{n-1} - Y_{n-2}}{6} . \quad (20)$$

К полученным уровням ряда динамики также можно применить метод скользящей средней.

3. Аналитическое выравнивание.

Предполагается, что трендовая модель имеет вид (21):

$$y_t = f(t) + \varepsilon_t , \quad (21)$$

где $f(t)$ – математическая функция, определяющая тенденцию развития; ε_t – случайное или циклическое отклонение от тенденции.

Целью аналитического выравнивания ряда динамики является определение аналитической или графической зависимости $f(t)$.

Чаще всего при выравнивании используются следующие зависимости:

- ✓ линейная $f(t) = a_0 + a_1 t$,
- ✓ параболическая $f(t) = a_0 + a_1 t + a_2 t^2$,
- ✓ экспоненциальная $f(t) = \exp(a_0 + a_1 t)$.

Оценка неизвестных параметров осуществляется методом наименьших квадратов (22):

$$\sum_{t=1}^n (y_t - f(t))^2 \rightarrow \min . \quad (22)$$

Один из подходов к упрощению расчетов заключается в переносе начала координат в середину ряда динамики. То есть, отсчет ведется от середины ряда динамики – условного нуля, при этом $\sum t_i = 0$.

При нечетном числе уровней средняя точка (например, год или месяц) принимается за 0. Тогда предшествующие периоды обозначаются соответственно: $-1, -2, -3$, а следующие за нулем $1, 2, 3$ и т.д.

При четном числе уровней два срединных момента (периода) времени обозначаются -1 и $+1$, а все последующие периоды, соответственно, через два интервала: $\pm 3, \pm 5, \pm 7$ и т.д.

Рассмотрим уравнение $f(t) = a_0 + a_1 t$

При переносе начала координат отсчета времени получаем формулы:

$$a_0 = \frac{\sum y_t}{n}, a_1 = \frac{\sum y_t t}{\sum t^2}.$$

Пример. Определим основную тенденцию ряда динамики числа продаж квартир в определенном регионе за 2010–2014 годы.

Годы	Число проданных квартир, тыс. ед. (y_t)	t	t ²	y*t
2010	108	-2	4	-216
2011	107	-1	1	-107
2012	110	0	0	0
2013	111	1	1	111
2014	112	2	4	224
Итого	548	0	10	12

$$a_0 = \frac{548}{5} = 109,6; a_1 = \frac{12}{10}; f(t) = 109,6 + 1,2t.$$

1.7 Кластерный анализ

Название кластерный анализ происходит от английского слова cluster – гроздь, скопление. Главное назначение кластерного анализа – разбиение множества исследуемых объектов и признаков на однородные в соответствующем понимании группы или кластеры.

В кластерном анализе считается, что:

- а) выбранные характеристики допускают в принципе желательное разбиение на кластеры;
- б) единицы измерения (масштаб) выбраны правильно.

Выбор масштаба играет большую роль. Как правило, данные нормализуют вычитанием среднего и делением на стандартное отклонение, так что дисперсия оказывается равной единице.

Решением задачи кластерного анализа являются разбиения, удовлетворяющие определенному критерию оптимальности. Этот критерий может представлять собой некоторый функционал, который называют целевой функцией. Например, в качестве целевой функции может быть взята внутригрупповая сумма квадратов отклонения (23):

$$W = \sigma_n = \sum_{j=1}^n (x_j - \bar{x})^2 = \sum_{j=1}^n x_j^2 - \frac{1}{n} \left(\sum_{j=1}^n x_j \right)^2. \quad (23)$$

где x_j - представляет собой измерения j -го объекта.

Кластер имеет следующие математические характеристики: *центр, радиус, среднеквадратическое отклонение, размер кластера*.

Центр кластера – это среднее геометрическое место точек в пространстве переменных.

Радиус кластера – максимальное расстояние точек от центра кластера. Кластеры могут быть перекрывающимися. В этом случае невозможно при помощи математических процедур однозначно отнести объект к одному из кластеров. Такие объекты называют спорными. *Спорный объект* - это объект, который может быть отнесен к нескольким кластерам. Неоднозначность данной задачи может быть устранена экспертом или аналитиком.

Размер кластера может быть определен либо по радиусу кластера, либо по среднеквадратичному отклонению объектов для этого кластера.

Методы кластерного анализа можно разделить на две группы:

- иерархические;
- неиерархические.

Каждая из групп включает множество подходов и алгоритмов. Используя различные методы кластерного анализа, аналитик может получить различные решения для одних и тех же данных.

Суть иерархической кластеризации состоит в последовательном объединении меньших кластеров в большие или разделении больших кластеров на меньшие.

Иерархические агломеративные методы (Agglomerative Nesting, AGNES) характеризуются последовательным объединением исходных элементов и соответствующим уменьшением числа кластеров.

В начале работы алгоритма все объекты являются отдельными кластерами. На первом шаге наиболее похожие объекты объединяются в кластер. На последующих шагах объединение продолжается до тех пор, пока все объекты не будут составлять один кластер.

Иерархические дивизимные (делимые) методы (DIvisive ANAlysis, DIANA) являются логической противоположностью агломеративным методам.

В начале работы алгоритма все объекты принадлежат одному кластеру, который на последующих шагах делится на меньшие кластеры, в результате образуется последовательность расщепляющих групп.

Принцип работы описанных выше групп методов в виде *дендрограммы* показан на рис. 11.

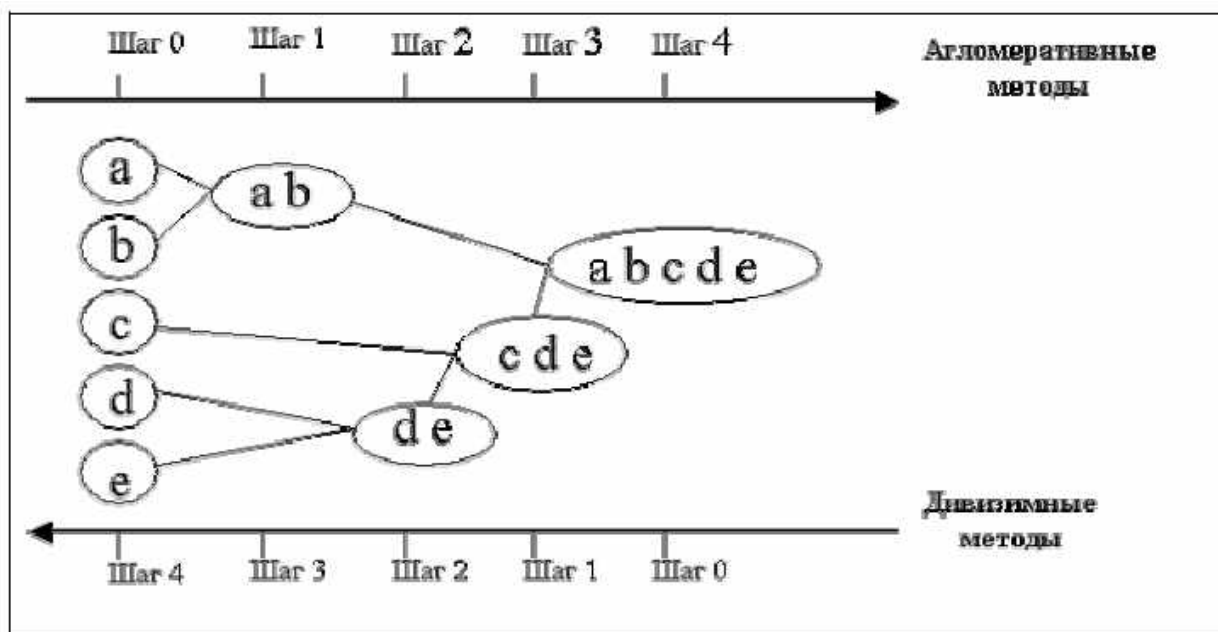


Рис. 11 – Дендрограмма иерархических методов

К неиерархическим методам классификации относится многочисленная группа так называемых *итеративных методов* кластерного анализа.

Сущность их заключается в том, что процесс классификации начинается с задания некоторых начальных условий (количество образуемых кластеров, порог завершения процесса классификации и т.д.).

К наиболее простым и эффективным алгоритмам кластеризации относится алгоритм *k-means* (*k*–средних). Он состоит из четырёх шагов:

1. Задаётся число кластеров *k*, которое должно быть сформировано из объектов исходной выборки.

2. Случайным образом выбирается *k* записей, которые будут служить начальными центрами кластеров.

3. Для каждой записи исходной выборки определяется ближайший к ней центр кластера.

4. Производится вычисление *центроидов* – центров тяжести кластеров. Это делается путём определения среднего для значений каждого признака всех записей в кластере.

Шаги 3 и 4 повторяются до тех пор, пока не будет выполнено условие в соответствии с некоторым критерием сходимости (чаще всего используется сумма квадратов ошибок между центроидом кластера и всеми вошедшими в него записями).

Остановка алгоритма также производится, когда на каждой итерации во всех кластерах остаётся один и тот же набор записей.

В основе следующего алгоритма – *G-means* – лежит предположение о том, что кластеризуемые данные подчиняются некоторому унимодальному закону распределения, например гауссовскому. Тогда центр кластера, определяемый как среднее значений признаков попавших в него объектов, может рассматриваться как мода соответствующего распределения.

Если исходные данные описываются унимодальным гауссовским распределением с заданным средним, то можно предположить, что все они относятся к одному кластеру.

Если распределение данных не гауссовское, то выполняется разделение на два кластера. Алгоритм G-means является *итеративным*, где на каждом шаге с помощью обычного алгоритма k-means строится модель с определённым числом кластеров (начальное значение k выбирается равным 1), которое увеличивается на каждой итерации. Увеличение k производится за счёт разбиения кластеров, в которых данные не соответствуют гауссовскому распределению.

Фактически в процессе работы G-means алгоритм k-means будет повторён k раз.

1.8 Дискриминантный анализ

С помощью дискриминантного анализа на основании некоторых признаков (независимых переменных) объект может быть причислен к одной из n заданных заранее групп. Такая постановка задачи, в особенности в случае двух групп, напоминает постановку задачи для метода логистической регрессии.

Ядром дискриминантного анализа является построение для каждой из n групп дискриминантной функции (24):

$$d = w_1x_1 + w_2x_2 + \dots + w_mx_m + w_0, \quad (24)$$

где $x_1, \dots, x_m$ – значения переменных, соответствующих классифицируемому объекту, константы $w_1, \dots, w_m$ и w_0 – коэффициенты одной из n групп, которые и предстоит оценить с помощью дискриминантного анализа. Целью является определение таких коэффициентов, чтобы по значениям дискриминантной функции можно было с максимальной четкостью провести разделение по группам.

Рассмотрим интерпретацию задач дискриминантного анализа в терминах нейронных сетей.

Общий вид искусственного нейрона (для одного линейного элемента) приведен на рис. 12.

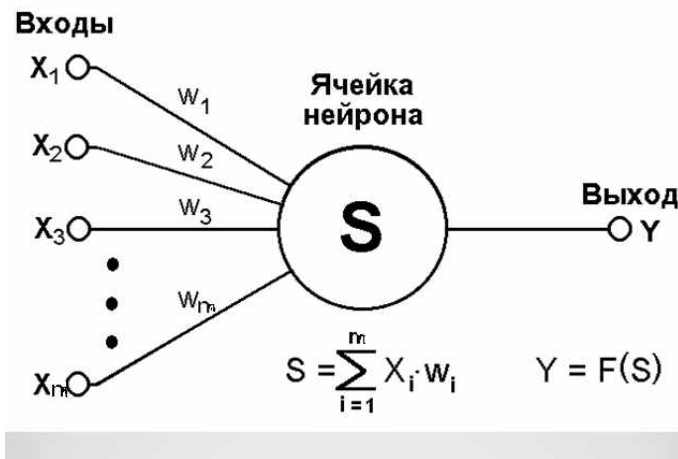


Рис. 12 – Общий вид искусственного нейрона

Нейронная сеть Кохонена: базовая версия.

Слой Кохонена состоит из некоторого количества n параллельно действующих линейных элементов. Это векторы w_j . Все они имеют одинаковое число входов m и получают на свои входы один и тот же вектор входных сигналов $x = (x_1, \dots, x_m)$. На выходе j -го линейного элемента получаем сигнал (25)

$$y_j = w_{j0} + \sum_{i=1}^m w_{ji} x_i, \quad (25)$$

где w_{ij} – весовой коэффициент i -го входа j -го нейрона, w_{j0} – пороговый коэффициент.

После прохождения слоя линейных элементов сигналы посылаются на обработку по правилу «победитель забирает всё»: среди выходных сигналов y_j ищется максимальный; его номер $j_{\max} = \arg \max_j \{y_j\}$.

Окончательно, на выходе сигнал с номером $j_{\max}$ равен единице, остальные – нулю.

«Нейроны Кохонена можно воспринимать как набор электрических лампочек, так что для любого входного вектора загорается одна из них».

Нейронная сеть Кохонена: геометрическая интерпретация.

Большое распространение получили слои Кохонена, построенные следующим образом: каждому j -му нейрону сопоставляется точка $W_j = (w_{j1}, \dots, w_{jm})$ в m -мерном пространстве (пространстве сигналов).

Для входного вектора $x = (x_1, \dots, x_m)$ вычисляются его евклидовы расстояния $\rho_j(x)$ до точек W_j и «ближайший получает всё» – тот нейрон, для которого это расстояние минимально, выдаёт единицу, остальные – нули.

Следует заметить, что для сравнения расстояний достаточно вычислять линейную функцию сигнала: $\rho_j(x)^2 = \|x - W_j\|^2$.

В качестве примера на рис. 13 приведено разбиение плоскости на многоугольники Вороного-Дирихле для случайно выбранных точек W_j при $m=2$ (каждая точка указана в своём многоугольнике):

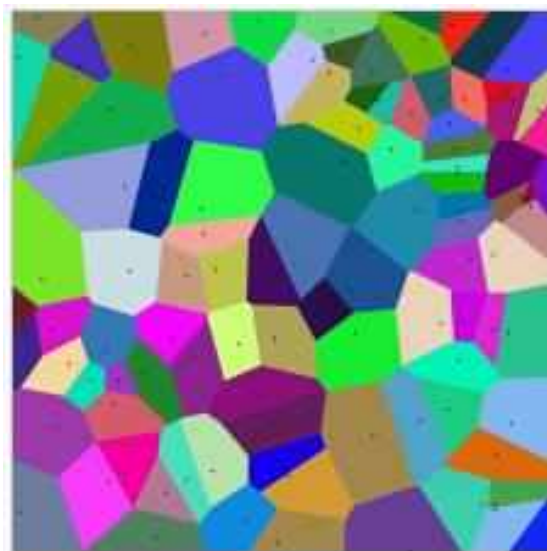


Рис. 13 – Разбиение плоскости на многоугольники Вороного-Дирихле

Определение и корректировка весовых коэффициентов сигнала называется обучением нейронной сети, которая может обучаться с учителем или без него.

При **обучении с учителем** для каждого обучающего входного объекта требуется знание правильного ответа или функции оценки качества ответа. Такое обучение называют управляемым. Нейронной сети предъявляются значения входных и выходных сигналов, а она по определенному алгоритму подстраивает веса синаптических связей. В процессе обучения производится корректировка весовых коэффициентов сети по результатам сравнения фактических выходных значений с входными, известными заранее.

При **обучении без учителя** раскрывается внутренняя структура данных или корреляции между объектами в наборе данных. Выходы нейронной сети формируются самостоятельно, а весовые коэффициенты изменяются по алгоритму, учитывающему только входные и производные от них сигналы. Это обучение называют также неуправляемым. В результате такого обучения объекты распределяются по категориям, при этом сами категории и их количество могут быть заранее неизвестны.

2 Основы компьютерного анализа данных

2.1 Введение в R

Прежде всего R – язык программирования для статистической обработки данных и работы с графикой, но в тоже время это свободная программная среда с открытым исходным кодом, развиваемая в рамках проекта **Полное название проекта** (GNU). R применяется везде, где нужна работа с данными. Это не только статистика в узком смысле слова, но и первичный анализ (графики, таблицы сопряжённости), и продвинутое математическое моделирование. R без особых проблем может использоваться и там, где сейчас принято использовать коммерческие программы анализа уровня MatLab/Octave. С другой стороны вполне естественно, что основная вычислительная мощь R лучше всего проявляется при статистическом анализе: от вычисления средних величин до вейвлет-преобразований временных рядов.

География использования R очень разнообразна. Трудно найти американский или западноевропейский университет, где бы не работали с R. Очень многие серьёзные компании (например, Boeing) устанавливают R для работы. R для статистиков – это действительно глобально.

Начало работы с R

1. Установить среду R.
2. Установить графическую оболочку RStudio.

Для того чтобы установить библиотеки расширений, необходимо набрать в консоли `install.packages("name")`, где "name" – название библиотеки. Полный список библиотек можно увидеть здесь: <http://cran.rstudio.com>, для просмотра по категориям нужно выбрать «CRAN Task Views».

Для установки библиотек необходимо наличие выхода в Интернет.

Для загрузки установленной библиотеки наберите в консоли `library(name)`.

Рабочая директория

Чтобы определить текущую рабочую директорию введите `getwd()`.

Для того чтобы изменить текущую директорию нужно в меню Session выбрать Set Working Directory/Choose Directory... и выбрать необходимую папку или прописать в консоли путь к папке: `setwd("c:/temp/")`.

Создание файлов

Для того чтобы создать текстовый файл в RStudio необходимо в меню File выбрать New File/Text File

Для создания файла с кодом выберите в меню файл New File/R script или нажмите Ctrl+Shift+N. Если вы уже выполнили несколько команд, то сохранить их в файл со скриптом можно следующим образом: `savehistory(file="myscript.r")`, где `myscript` – название файла.

Сочетания клавиш

- ✓ Ввод предыдущей команды – «↑»
- ✓ Очистить консоль – Ctrl+L
- ✓ Выполнить выделенный код (для выполнения строки кода в файлах с расширениями .R и .Rmd) – Ctrl+Enter

Справка

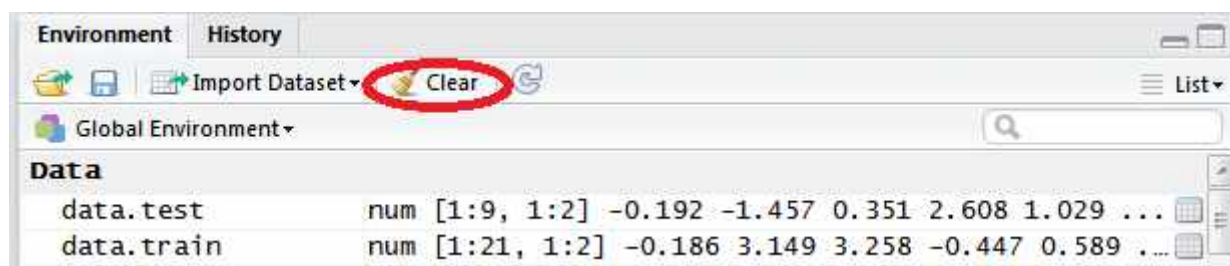
Для того чтобы узнать как правильно вызывать функцию следует воспользоваться встроенной справкой. Для этого наберите `help(name)` или `?name`, где `name` – это название функции.

Так же для вызова справки можно воспользоваться командой `help.start()` или выбрать в меню Help/R help. В этом случае откроется окно, в котором будет демонстрироваться HTML-помощь. Ей удобно пользоваться в случае, если вы не знаете точного названия необходимой функции.

Рабочее пространство

Посмотреть список всех используемых переменных можно так: `ls()`

Удалить переменную можно, воспользовавшись командой `rm(name)`, где `name` – это имя переменной. Удалить все переменные рабочего пространства можно следующим образом: `rm(list=ls())` или в меню рабочей среды нажать кнопку Clear:



2.2 Форматы данных

Числовые векторы

Для создания числового вектора необходимо воспользоваться функцией `c()`:

```
x<-c(1,2,3,4,5)
```

где `x` – это имя объекта R, `<-` – функция присвоения, `c()` – функция создания вектора. Собственно, R и работает в основном с объектами и функциями. У объекта может быть своя структура:

```
> x<-c(1,2,3,4,5)
```

```
> str(x)
num [1:5] 1 2 3 4 5
```

то есть `x` – это числовой вектор. В языках программирования бывают еще скаляры, но в R скаляров нет. Одиночные объекты трактуются как векторы из одного элемента. Вот так можно проверить вектор перед нами или нет:

```
> is.vector(x)
[1] TRUE
```

Числовой вектор можно создать с помощью последовательности `seq`, где первая цифра говорит с какого числа начинается последовательность (-1), вторая – до какого числа (1), а третья – шаг (0.1):

```
> v1<-seq(-1,1,0.1)
> v1
[1] -1.0 -0.9 -0.8 -0.7 -0.6 -0.5 -0.4 -0.3 -0.2 -0.1  0.0  0.1  0.2  0.3
0.4  0.5
[17]  0.6  0.7  0.8  0.9  1.0
> str(v1)
num [1:21] -1 -0.9 -0.8 -0.7 -0.6 -0.5 -0.4 -0.3 -0.2 -0.1 ...
```

Можно и так:

```
> v2<-seq(from=-1,to=1,by=0.1)
> v2
[1] -1.0 -0.9 -0.8 -0.7 -0.6 -0.5 -0.4 -0.3 -0.2 -0.1  0.0  0.1  0.2  0.3
0.4  0.5
[17]  0.6  0.7  0.8  0.9  1.0
```

Или вот так:

```
> v3<-seq(length=21,from=-1,by=0.1)
> v3
[1] -1.0 -0.9 -0.8 -0.7 -0.6 -0.5 -0.4 -0.3 -0.2 -0.1  0.0  0.1  0.2  0.3
0.4  0.5
[17]  0.6  0.7  0.8  0.9  1.0
```

В случае если необходимо создать вектор из повторяющихся элементов, можно воспользоваться функцией `rep`:

```
> x<-c(1,2,3,4,5)
> rep(x,3) #элементы вектора x повторяются 3 раза
[1] 1 2 3 4 5 1 2 3 4 5 1 2 3 4 5
> rep(x,times=3) #элементы вектора x повторяются 3 раза
[1] 1 2 3 4 5 1 2 3 4 5 1 2 3 4 5
> rep(x,each=4) #каждый элемент вектора x повторяется 4 раза
[1] 1 1 1 1 2 2 2 2 3 3 3 3 4 4 4 4 5 5 5 5
```

Называть объекты можно в принципе как угодно, но лучше придерживаться некоторых простых правил:

- ✓ Использовать для названий только латинские буквы, цифры и точку (имена объектов не должны начинаться с точки или цифры);
- ✓ Помнить, что R чувствителен к регистру, `X` и `x` – это разные имена;
- ✓ Не давать объектам имена, уже занятые распространёнными функциями (типа `c()`), а также ключевыми словами (особенно `T`, `F`, `NA`, `NaN`, `Inf`).

Факторы

Для обозначения категориальных данных в R есть несколько способов, разной степени правильности. Во-первых, можно создать текстовый (`character`) вектор:

```
> spring_months<-c("march","april","may")
> is.character(spring_months)
[1] TRUE
```

```
> is.vector(spring_months)
[1] TRUE
> str(spring_months)
chr [1:3] "march" "april" "may"
```

Пропущенные векторы

В дополнение к векторам из чисел и текстовым векторам, R поддерживает ещё и логические вектора, а также специальные типы данных, которые бывают очень важны для статистических расчетов. Прежде всего это пропущенные или отсутствующие данные, которые обозначаются как NA.

```
> h<-NA
> is.vector(h)
[1] TRUE
> str(h)
logi NA
```

Как видно из примера, NA нужно вводить без кавычек и R понимает такие данные как логические переменные.

Матрицы

Матрица в R – это просто специальный тип вектора, обладающий некоторыми дополнительными свойствами, позволяющими интерпретировать его как совокупность строк и столбцов. Например, создадим матрицу 2X3 с помощью числового вектора:

```
> m<-1:6
> m
[1] 1 2 3 4 5 6
> matr <- matrix(m, ncol=3, byrow=TRUE)
> matr
      [,1] [,2] [,3]
[1,]    1    2    3
[2,]    4    5    6
> str(matr)
int [1:2, 1:3] 1 4 2 5 3 6
> str(m)
int [1:6] 1 2 3 4 5 6
```

Из примера видно, что структура объектов m и matr не слишком различается. Различается, по сути, лишь их вывод на экран. Обратиться к элементу матрицы можно так:

```
> matr[1,2]
[1] 2
> m[3]
[1] 3
```

Так же можно создавать матрицы путем объединения столбцов cbind() или строк rbind(), например:

```
> m1<-1:5
> m2<-c(1,4,3,5,7)
> cbind(m1,m2)
      m1 m2
[1,]   1  1
[2,]   2  4
[3,]   3  3
[4,]   4  5
[5,]   5  7
> rbind(m1,m2)
      [,1] [,2] [,3] [,4] [,5]
m1      1    2    3    4    5
```

m2 1 4 3 5 7

Списки

Список – это своеобразный вектор, элементами которого могут быть какие угодно структуры. Если вектор и матрица могут состоять из элементов только одного и того же типа, то список – из чего угодно, в том числе из других списков. Например:

```
> l <- list("R", 1:3, TRUE, NA, list("r", 4))
> l
[[1]]
[1] "R"

[[2]]
[1] 1 2 3

[[3]]
[1] TRUE

[[4]]
[1] NA

[[5]]
[[5]][[1]]
[1] "r"

[[5]][[2]]
[1] 4
```

Обратиться к элементу списка можно следующим образом:

```
> l[1]
[[1]]
[1] "R"
```

В этом случае результат будет тоже списком. Если обратиться так:

```
> l[[1]]
[1] "R"
```

то результат будет объектом того типа, которого он был до объединения в список. Так же можно использовать имена элементов списка, но для этого сначала их надо присвоить:

```
> l <- list("R", 1:3, TRUE, NA, list("r", 4))
> names(l) <- c("first", "second", "third", "fourth", "fifth")
> l$first
[1] "R"
> str(l$first)
chr "R"
```

Для выбора по имени используется \$, а полученный объект будет таким же, как при использовании [[]]. На самом деле имена в R могут иметь и элементы вектора, и строки, и столбцы матрицы:

```
> x
[1] 1 2 3 4 5
> names(x) <- c("x1", "x2", "x3", "x4", "x5")
> x
x1 x2 x3 x4 x5
1 2 3 4 5
> matr
      [,1] [,2] [,3]
[1,]    1    2    3
[2,]    4    5    6
> rownames(matr) <- c("a1", "a2")
> colnames(matr) <- c("b1", "b2", "b3")
> matr
      b1 b2 b3
```

```
a1 1 2 3
a2 4 5 6
```

Единственное условие состоит в том, что все имена должны быть разными. Однако, знак \$ можно использовать только со списками. К элементам вектора и матрицы по имени можно обратиться следующим образом:

```
> x["x3"]
x3
3
> matr["a1","b2"]
[1] 2
```

Таблицы данных

Таблицы данных больше всего похожи на электронные таблицы Excel и аналогов, и поэтому с ними работают чаще всего. Каждая таблица данных – это список колонок, причём внутри одной колонки все данные должны быть одного типа.

Например, пусть имеются данные выборки *v* по которым необходимо построить дискретный вариационный ряд. Для этого применим к данным функцию `table`. Она возвращает таблицу, состоящую из вектора количеств встреч всех различных значений в векторе *v* и вектора наименований всех различных значений в векторе *v* типа `character`. Для того чтобы получить числовой вектор всех различных значений вектора *v* воспользуемся функциями `as.numeric` (преобразовывает в числовой формат) и `attr` (возвращает соответствующий атрибут таблицы, в нашем случае не обходимо название столбцов, поэтому `"names"`). Далее преобразовываем данные таблицы в векторный формат, для этого используем функцию `as.vector` и с помощью функции `data.frame` строим таблицу данных, состоящую из двух колонок: наименование признака и количество):

```
> v<-c(16, 16, 16, 15, 17, 19, 16, 15, 14, 16, 17, 14, 17, 19, 16, 15, 17,
19, 14, 16, 14, 16, 19, 17, 18, 15, 16, 15, 16, 18)
> t<-table(v)
> t
v
14 15 16 17 18 19
 4  5 10  5  2  4
> str(t)
' table' int [1:6(1d)] 4 5 10 5 2 4
- attr(*, "dimnames")=List of 1
..$ v: chr [1:6] "14" "15" "16" "17" ...
> name<-as.numeric(attr(t,"names"))
>
> name
[1] 14 15 16 17 18 19
> str(name)
num [1:6] 14 15 16 17 18 19
> zn<-as.vector(t)
> zn
[1] 4 5 10 5 2 4
> data<-data.frame("Naimenovanie priznaka"=name,"kolichestvo"=zn)
> data
Naimenovanie.priznaka kolichestvo
1                      14          4
```

```

2          15          5
3          16         10
4          17          5
5          18          2
6          19          4
> str(data)
'data.frame': 6 obs. of 2 variables:
 $ Naimenovanie.priznaka: num 14 15 16 17 18 19
 $ kolichestvo          : int 4 5 10 5 2 4

```

Поскольку таблица данных является списком, к ней применимы методы индексации списков. Кроме того, таблицы данных можно индексировать и как двумерные матрицы:

```

> data$Naimenovanie
[1] 14 15 16 17 18 19
> data[1]
Naimenovanie.priznaka
1          14
2          15
3          16
4          17
5          18
6          19
> data[[1]]
[1] 14 15 16 17 18 19
> data[,2]
[1] 4 5 10 5 2 4
> data[1,1]
[1] 14

```

Добавить столбец в таблице данных можно просто указав другое название столбца. Например, добавим столбец с подсчитанной накопленной частотой:

```

> data["Nakoplennaya chastota"]<-cumsum(zn)
> data
Naimenovanie.priznaka kolichestvo Nakoplennaya chastota
1          14          4          4
2          15          5          9
3          16         10         19
4          17          5         24
5          18          2         26
6          19          4         30

```

Векторизованные вычисления

Одна из особенностей R заключается в том, что если применить какую-то арифметическую операцию ко всем элементам вектора, то не нужно обращаться к каждому элементу отдельно, достаточно указать сразу весь вектор. Например:

```

> v
[1] 16 16 16 15 17 19 16 15 14 16 17 14 17 19 16 15 17 19 14 16 14 16 19 17
18
[26] 15 16 15 16 18
> v*10
[1] 160 160 160 150 170 190 160 150 140 160 170 140 170 190 160 150 170 190
140
[20] 160 140 160 190 170 180 150 160 150 160 180

```

Векторизованы и операции между векторами и матрицами:

```

> m1<-1:4
> m1

```



```

[1] 1 2 3 4
> m2<-4:7
> m2
[1] 4 5 6 7
> m1+m2
[1] 5 7 9 11
> matr1 <- matrix(m1, ncol=2, byrow=TRUE)
> matr1
      [,1] [,2]
[1,]    1    2
[2,]    3    4
> matr2<- matrix(m2, ncol=2, byrow=TRUE)
> matr2
      [,1] [,2]
[1,]    4    5
[2,]    6    7
> matr1+matr2
      [,1] [,2]
[1,]    5    7
[2,]    9   11
> matr1*matr2
      [,1] [,2]
[1,]    4   10
[2,]   18   28
> matr1%%matr2 #матричное произведение
      [,1] [,2]
[1,]   16   19
[2,]   36   43

```

Помимо простых арифметических операций, векторизованы и более сложные преобразования. Есть целое семейство функций, которые специализируются на векторизованных вычислениях: `apply`, `by`, `lapply`, `sapply`, `tapply` и другие. Вот как работает, например, функция `apply`:

```

> data
  Naimenovanie.priznaka  kolichestvo
1                   14              4
2                   15              5
3                   16             10
4                   17              5
5                   18              2
6                   19              4
> apply(data, 2, sum)
  Naimenovanie.priznaka      kolichestvo
                   99                   30

```

В таблице данных `data` для 2-х столбцов вычисляется сумма всех элементов (`sum`).

Чтение данных из файла

Данные, разделенные запятой, из файла можно прочитать следующим образом:

```
dat<- read.table(f.txt", sep=",", header=T)
```

В результате создается таблица `dat` в строках которой по записям размещены данные из файла `f.txt` которые разделены разделителем (`sep=","` – в данном случае запятой, если `sep="\t"` – табуляцией и т.д. В случае если кроме пробелов разделителей никаких нет, то можно не указывать), причём первая запись воспринимается как заголовок (`header=T`, если заголовка нет, то можно не указывать), то есть содержит названия столбцов тоже разделённые

указанным разделителем, если строка начинается с символа # то она игнорируется как комментарий.

Записать данные таблицы в файл можно следующим образом:

```
write.table(data,"f.txt",sep=",")
```

где data – таблица, которую необходимо записать, sep="," – разделитель с которым данные будут записаны (в данном случае через запятую, если только через пробел, то разделитель можно не указывать)

2.3 Управляющие конструкции и функции

Условия

Условия можно оформлять несколькими способами:

1) В виде: if (условие) команда1 else команда2. Команда1 выполняется, если условие дало результат True, иначе выполняется команда2. Например:

```
> x<-10
> if(x<10) y<-x+1 else y<-x-1
> y
[1] 9
```

Если нужно выполнить сразу несколько команд:

```
> x<-10
> if (x<10) {y<-x+1
+           prin(y)} else
+           {y<-x-1
+           print(y)}
[1] 9
```

2) В виде: ifelse(условие, команда1, команда2). Команда1 выполняется, если условие дало результат True, иначе выполняется команда2.

```
> x<-10
> ifelse(x<10,y<-x+1,y<-x-1)
[1] 9
```

Условия можно применять к векторам. Например:

```
> v<-c(16, 16, 16, 15, 17, 19, 16, 15, 14, 16, 17, 14, 17, 19, 16, 15, 17,
19, 14, 16, 14, 16, 19, 17, 18, 15, 16, 15, 16, 18)
> ifelse(v>=17,T,F)
[1] FALSE FALSE FALSE FALSE TRUE TRUE FALSE FALSE FALSE FALSE TRUE FALSE
TRUE
[14] TRUE FALSE FALSE TRUE TRUE FALSE FALSE FALSE FALSE TRUE TRUE TRUE
FALSE
[27] FALSE FALSE FALSE TRUE
```

Циклы

Как и в языках программирования в R есть циклы.

1) Цикл for:

```
> s<-0
> for (i in 1:10){ s=s+i }
> s
[1] 55
```

2) Цикл while:

```
> x<-10
> while(x>0) {x<-x-1}
> x
[1] 0
```

Функции

Общий вид структуры функции следующий:

```
function( arglist ) expr
return(value)
```

Где `arglist` — список входных параметров функции, `expr` — выражения выполняемые в функции, `value` — список выходных параметров функции.

Например:

```
> data.mean<-function(data)
+   {if(!is.data.frame(data)) stop("Invalid data")} #проверка
входные данные – это таблица
+ Sr<-sum(data$Naimenovanie_priznaka*data$kolichestvo)/sum(data[,2])
#вычисляет среднее значение вариационного ряда
+ return(Sr)}
> data.mean(data)
[1] 16.26667
> data.mean(v) # если передать вектор v, то будет ошибка
Error in data.mean(v) : Invalid data
```

2.4 Статистические функции

Для того чтобы сформировать случайную подвыборку из 10 элементов вектора `v` воспользуемся функцией `sample`:

```
> v<-c(16, 16, 16, 15, 17, 19, 16, 15, 14, 16, 17, 14, 17, 19, 16, 15, 17,
19, 14, 16, 14, 16, 19, 17, 18, 15, 16, 15, 16, 18)
> sample(v,10)
[1] 16 16 17 16 16 14 14 18 17 15
```

Случайная расстановка элементов вектора `v`:

```
> sample(v)
[1] 19 17 16 16 18 14 19 16 16 18 16 16 15 17 14 15 17 19 17 17 16 15 14 16
16
[26] 15 16 19 15 14
```

Частота появления различных значений элементов вектора `v`:

```
> table(v)
v
14 15 16 17 18 19
 4  5 10  5  2  4
```

Количество элементов вектора `v`:

```
> length(v)
[1] 30
```

Вычислить среднее значение вектора `v`:

```
> mean(v)
[1] 16.26667
```

Вычислить выборочную дисперсию для элементов вектора `v`:

```
> var(v)
[1] 2.34023
```

Вычислить среднее квадратическое отклонение вектора `v`:

```
> sd(v)
[1] 1.529781
```

Вычислить медиану вектора `v`:

```
> median(v)
[1] 16
```

Вычислить моду вектора `v` можно используя построенный дискретный вариационный ряд:

```
> data
  Naimenovanie.priznaka kolichestvo
1                   14             4
2                   15             5
3                   16            10
4                   17             5
5                   18             2
```

```

6                               19          4
> data[[1]][which.max(data[,2])] #мода
[1] 16

```

Вычислить коэффициент корреляции между элементами векторов v и

w :

```

> v<-c(48,52,51,47,49,54,46,49,50,46,47,47,52,44,48,52,52,45)
> w<-c(34,24,36,33,23,24,35,30,30,33,32,31,24,36,33,27,27,34)
> cor(v,w)
[1] -0.7439125

```

Выполнить кумулятивное суммирование можно следующим образом:

```

> zn
[1] 4 5 10 5 2 4
> cumsum(zn)
[1] 4 9 19 24 26 30

```

Построить линейную регрессионную модель вида $y=a+b*x$ по набору данных, содержащему переменные x и y , можно таким образом:

```
lm(y~x)
```

Построить регрессионную модель вида $y=a+b*x_1+c*x_2$ по набору данных, содержащему переменные x_1 , x_2 и y , можно таким образом:

```
lm(y~x1+x2)
```

Случайные величины

Сгенерировать случайные величины с равномерным распределением можно с помощью функции `runif`:

```

> runif(10,-5,5) #10-количество генерируемых случайных величин на отрезке [-5,5]
[1] 2.6091505 0.7736650 1.9701312 1.6586380 -3.7601478 0.6622884 -
1.5718614
[8] 1.0735858 -4.6243254 2.6693043

```

Генерация случайных величин с биномиальным распределением:
 > `rbinom(10,1,0.5)` #10 – количество генерируемых случайных величин, 1-число испытаний, 0.5 – вероятность успеха в каждом испытании

```

[1] 1 1 0 1 0 1 0 1 1 0

```

Генерация случайных величин со стандартным нормальным распределением (математическое ожидание равно 0, дисперсия равна 1):

```

> rnorm(10)
[1] -0.37724887 -0.65809596 -2.11901103 0.85971433 -1.01672943 -1.46440212
[7] -0.76999775 -0.07545189 -0.27882906 -1.10841932

```

2.5 Графики

С помощью функции `plot(x,y)` можно строить графики. В случае если x и y числовые векторы:

```

> v
[1] 16 16 16 15 17 19 16 15 14 16 17 14 17 19 16 15 17 19 14 16 14 16 19 17
18 15 16
[28] 15 16 18
> data
  Naimenovanie.priznaka  kolichество  Nakoplennaya  chastota
1                    14             4                4
2                    15             5                9
3                    16            10               19
4                    17             5               24
5                    18             2               26
6                    19             4               30
> plot(data[,1],data[,2],type='l',main="Полигон частот",xlab="наименование
признака", ylab="частота") # type='l'– тип соединения (в данном случае –

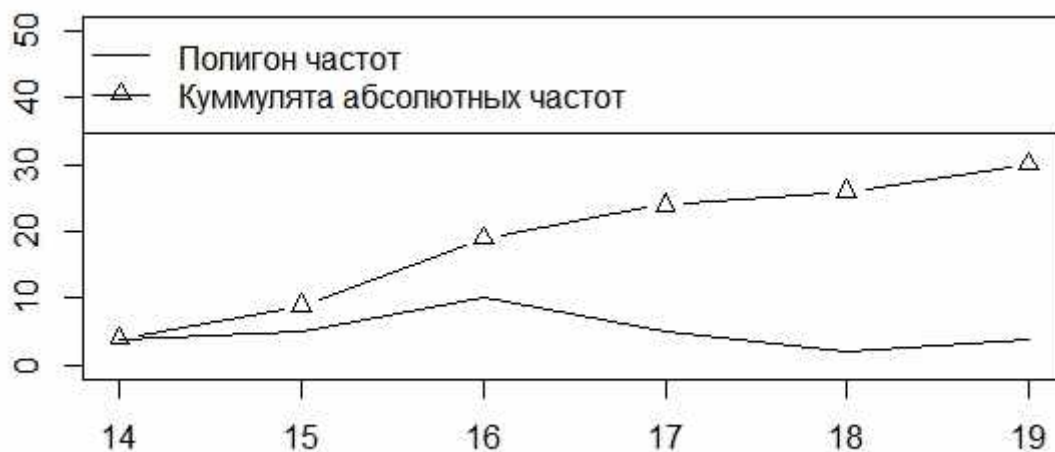
```

линия), main – заголовок графика, xlab – заголовок оси x, ylab – заголовок оси y



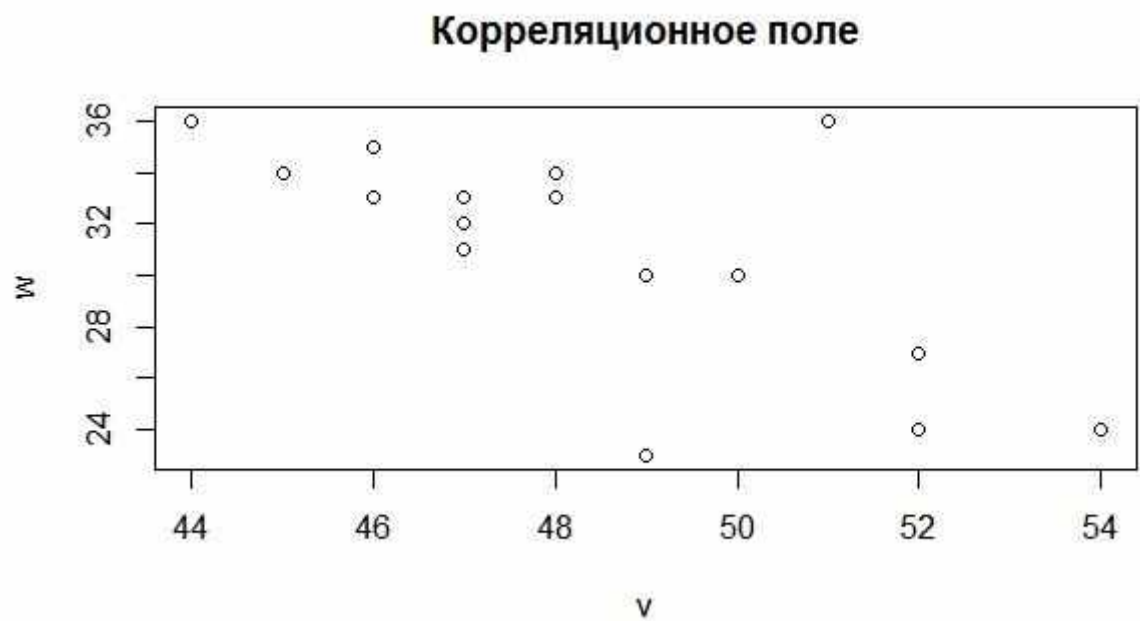
Построим на одном графике полигон частот и куммуляту абсолютных частот (для этого воспользуемся функцией points (график изобразится точками) или lines(график изобразится линиями):

```
> plot(data[,1],data[,2],type='l',ylim=c(0,max(data[,3])+20),xlab=NA,ylab=NA)
> points(data[,1],data[,3],type='b',pch=2) # type='b' означает и линиями, и
точками, pch=2 – фигура для точек (в данном случае треугольник)
> legend("topleft", legend=c("полигон частот","кумулята абсолютных частот"),
lty=c(1,1),pch=c(NA,2)) #легенда
```

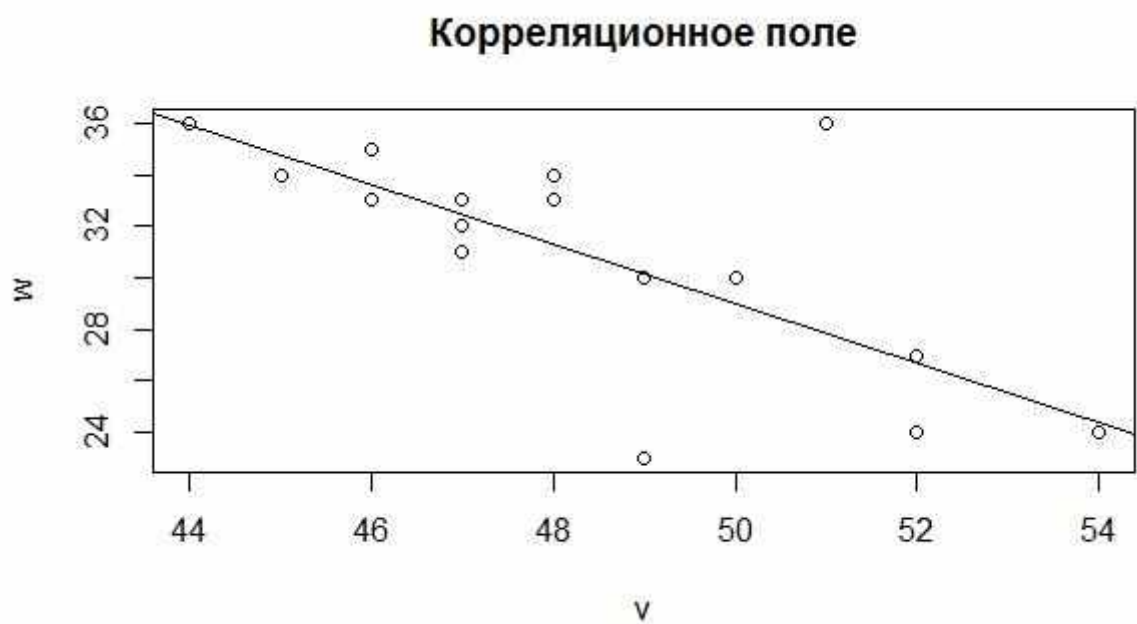


Построим корреляционное поле:

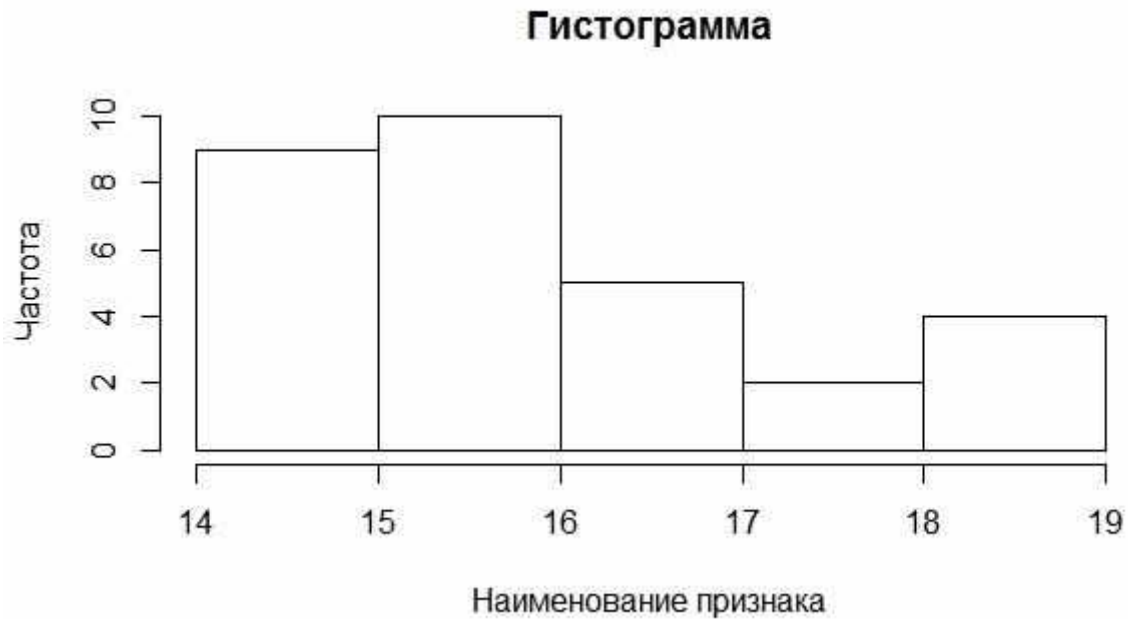
```
> v<-c(48,52,51,47,49,54,46,49,50,46,47,47,52,44,48,52,52,45)
> w<-c(34,24,36,33,23,24,35,30,30,33,32,31,24,36,33,27,27,34)
> plot(v,w,main="Корреляционное поле")
```



Добавим на корреляционное поле линию регрессии:
`> abline(lm(w~v))`

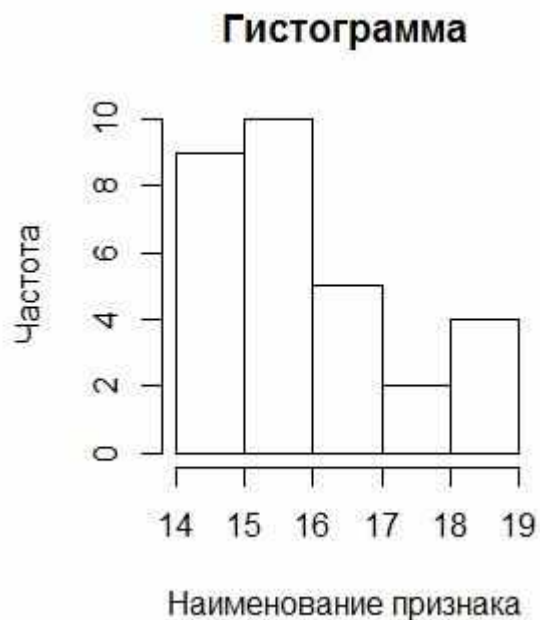
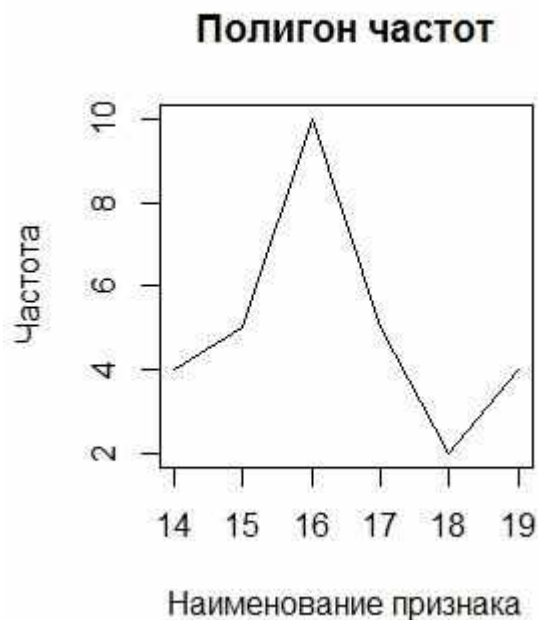


Построим гистограмму вектора v:
`> hist(v,main="Гистограмма",xlab="Наименование признака", ylab="частота")`



Нарисовать несколько графиков в одном окне можно так:

```
> par(mfrow=c(1,2)) # количество отображаемых графиков: 1 строка, 2 столбца
> plot(data[,1],data[,2],type='l',main="Полигон частот",xlab="Наименование признака", ylab="частота") # type='l' - тип соединения (в данном случае - линия), main - заголовок графика, xlab - заголовок оси x, ylab - заголовок оси y
> hist(v,main="Гистограмма",xlab="наименование признака", ylab="частота")
```



2.6 Кластерный анализ

Пусть имеются некоторые данные:

```
> dat
  v1 v2
1 48 34
```

```

2  52 24
3  51 36
4  47 33
5  49 23
6  54 24
7  46 35
8  49 30
9  50 30
10 46 33
11 47 32
12 47 31
13 52 24
14 44 36
15 48 33
16 52 27
17 52 27
18 45 34

```

Применим к ним алгоритм кластерного анализа «К-средних» (kmeans). Алгоритм требует явного задания количества кластеров (классов). Разобьём исходные данные на 2 кластера:

```

> cl<-kmeans(dat,2)
> cl
к-means clustering with 2 clusters of sizes 6, 12

```

```

Cluster means:
      v1      v2
1 51.83333 24.83333
2 47.33333 33.08333

```

```

Clustering vector:
[1] 2 1 2 2 1 1 2 2 2 2 2 2 1 2 2 1 1 2

```

```

within cluster sum of squares by cluster:
[1] 27.66667 91.58333
(between_ss / total_ss = 74.8 %)

```

Available components:

```

[1] "cluster"      "centers"      "totss"      "withinss"
"tot.withinss"
[6] "betweenss"    "size"        "iter"      "ifault"

```

```

> cl$centers #центры кластеров

```

```

      v1      v2
1 51.83333 24.83333
2 47.33333 33.08333

```

```

> cl$cluster #результат кластеризации
[1] 2 1 2 2 1 1 2 2 2 2 2 2 1 2 2 1 1 2

```

```

> table(cl$cluster) #показывает сколько данных попало в первый класс, а
сколько во второй

```

```

 1  2
 6 12

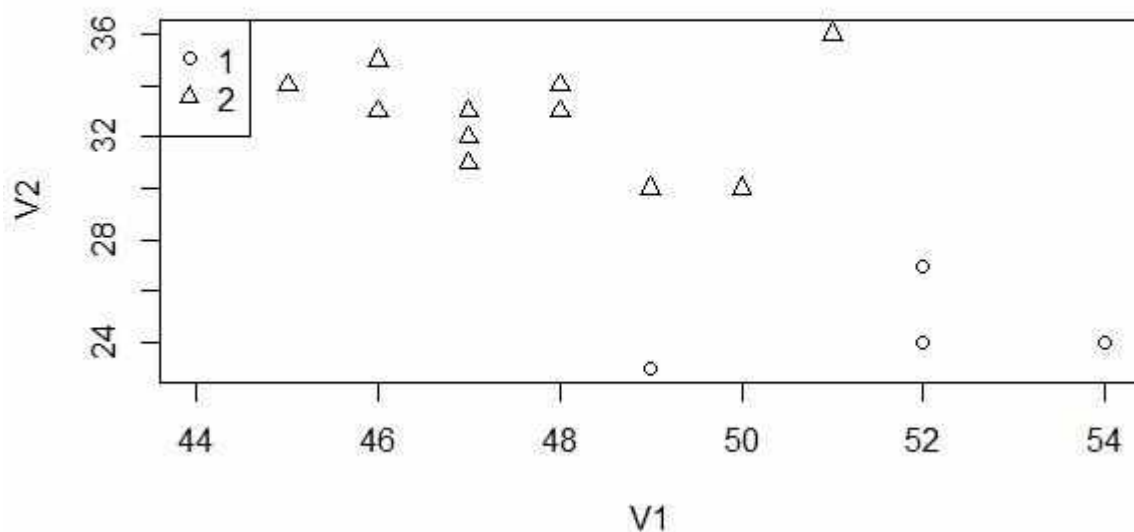
```

```

Построим график, отражающий классификацию данных:
> plot(dat,col=ifelse(cl$cluster==1,"blue","green")) #отображение различными
цветами
> legend("topleft",legend=c("1","2"),fill=c("blue","green"))

> plot(dat,pch=ifelse(cl$cluster==1,1,2)) #отображение с помощью фигур
> legend("topleft",legend=c("1","2"),pch=c(1,2))

```

2.7 Дискриминантный анализ

Для проведения дискриминантного анализа будем использовать функции из библиотеки MASS, подключить которую можно следующим образом:

```
> library(MASS)
```

Сгенерируем двумерные данные с нормальным распределением:

```
> x<-c(rnorm(10),rnorm(20)+2)
> y<-c(rnorm(10),rnorm(20)+2)
> xy<-cbind(x,y)
> xy
```

	x	y
[1,]	-1.5300419	-1.01769797
[2,]	-0.3122608	0.31839523
[3,]	1.1621753	0.84706067
[4,]	-1.4593408	-0.21226105
[5,]	-0.3145371	0.36138263
[6,]	0.2138121	-0.80870903
[7,]	1.6090436	-1.27245688
[8,]	0.1613994	1.63051543
[9,]	-2.1936009	0.06914084
[10,]	-1.3219576	-2.28891183
[11,]	0.5974567	1.26202503
[12,]	1.7646661	3.37932167
[13,]	2.7743485	3.17290476
[14,]	-0.7925304	3.76069004
[15,]	2.2303363	1.67884680
[16,]	2.5736544	3.61097133
[17,]	2.9650799	0.08916613
[18,]	1.9115747	2.10025807
[19,]	1.7858393	1.20534584
[20,]	0.5264816	1.18018846
[21,]	3.0036779	3.05776805
[22,]	3.0521164	2.21359636
[23,]	2.5518334	1.82746648
[24,]	1.7809334	3.18431120
[25,]	1.8770278	0.17437605
[26,]	0.9850558	-0.45052819
[27,]	2.9529567	0.98610936

```

[28,] 1.6936971 2.78140222
[29,] 1.7898521 3.86385170
[30,] 2.1497643 2.48798285

> n<-length(xy[,1]) #количество данных
> n
[1] 30
> n.train<-floor(n*0.7) #количество наблюдений для обучения: 70%
> n.test<-n-n.train #количество наблюдений для тестирования: 30%
> idx.train<-sample(1:n,n.train) #случайный выбор индексов для обучающей
выборки
> data.train<-xy[idx.train,] #из данных xy выбирает те из них, которые
соответствуют случайному выбору индексов
> idx.test<-(1:n)[!(1:n %in% idx.train)] #выбирает оставшиеся наблюдения в
тестовую выборку
> data.test<-xy[idx.test,] #из данных xy выбирает оставшиеся наблюдения в
тестовую выборку

```

Для того чтобы получить вектор истинной классификации выполняем кластерный анализ данных xy:

```

> cl<-kmeans(xy,2)
> cl.xy<-cl$cluster
> cl.xy #классификация данных xy
[1] 2 2 2 2 2 2 2 2 2 2 1 1 1 1 1 1 2 1 1 1 1 2 2 1 1 1 1
> cl.train<-cl.xy[idx.train]
> cl.train #классификация данных обучающей выборки
[1] 1 1 2 1 1 1 2 1 2 2 2 1 1 2 2 2 1 2 2 2 1
> cl.test<-cl.xy[idx.test] #классификация данных тестовой выборки
> cl.test
[1] 2 2 1 1 1 1 2 1 1

```

Проводим обучение полученной обучающей выборки:

```

> mod<-qda(data.train,cl.train)

```

Для классификации тестовых данных на основе полученной в результате обучения закономерности воспользуемся функцией predict:

```

> cl.test_est<-predict(mod,data.test)$class
> cl.test_est #полученная классификация тестовой выборки
[1] 2 2 1 1 1 2 2 1 1
Levels: 1 2

```

Сравним полученные результаты cl.test_est классификации с истинной классификацией тестовой выборки cl.test:

```

> sum(cl.test_est!=cl.test)/n.test
[1] 0.1111111
> idx<-idx.test[cl.test_est!=cl.test]
> idx #индексы тестовых данных, которые не соответствуют истинной
классификации
[1] 19

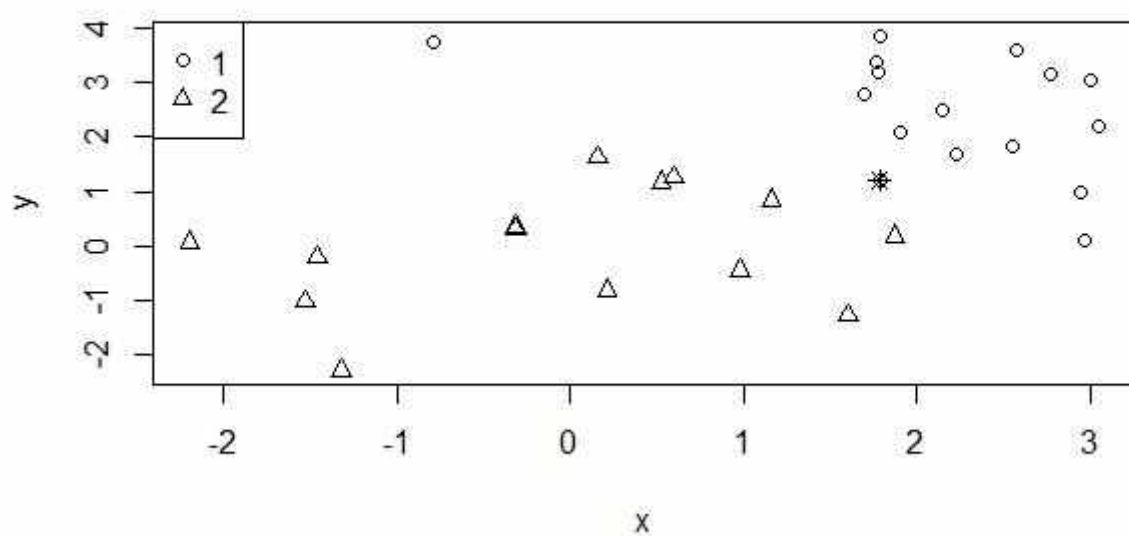
```

Построим график, на котором данные первого класса обозначены кружком, данные второго класса – треугольником, неверно классифицированные данные в результате дискриминантного анализа – звездочкой:

```

> plot(xy,col=ifelse(cl.xy==1,"blue","green"))
> legend("topleft",legend=c("1","2"),fill=c("blue","green"))
> points(xy[idx,][[1]],xy[idx,][[2]],col="red")

```



2.8 Лабораторные работы

Тексты индивидуальных лабораторных работ находятся в локальной сети БГУ по адресу: FPMI-STUD\subfaculty\KTC\Казаченок\ВКИАД